

## Atividade Individual — Análise do Mini C Compiler

### Lucas Rocha Dantas - 2311204 - Compiladores

**Repositório analisado:** [ironrinox/mini-c-compiler](https://github.com/ironrinox/mini-c-compiler)

**Objetivo:** o objetivo da atividade é analisar o funcionamento do projeto Mini C Compiler, acompanhando o fluxo desde a leitura do arquivo-fonte até a execução do programa. A análise envolve as etapas de leitura, análise léxica, geração de tokens, análise sintática, construção da AST e interpretação, além da identificação de limitações e implementação de uma melhoria individual.

**Preparação do ambiente:** a atividade foi realizada no Ubuntu, utilizando o terminal para manipulação dos arquivos, compilação e execução do projeto. O ambiente já possuía Git e GCC instalados, atendendo aos requisitos necessários para obtenção, compilação e execução do código.

**Compilação e execução:** o projeto foi compilado utilizando o GCC, que reuniu os arquivos da pasta src e os cabeçalhos da pasta include para gerar o executável mini-c. Após a compilação, o programa foi executado pelo terminal, informando como argumento o caminho do arquivo de teste. Durante a execução, são apresentados o código-fonte lido, os tokens gerados, a AST construída e o resultado da interpretação.

**Arquitetura dos arquivos:** o projeto é dividido em arquivos com responsabilidades específicas. src/main.c coordena o fluxo principal; src/utls.c realiza a leitura do arquivo-fonte; include/lexer.h define os tokens e src/lexer.c realiza a análise léxica; include/parser.h define a estrutura da AST e src/parser.c realiza sua construção; include/interpreter.h define a tabela de símbolos e as funções do interpretador, enquanto src/interpreter.c percorre a AST e executa o programa.

## Etapa 1 — Análise do ponto de entrada

### 1. Como o caminho do arquivo-fonte é recebido?

Pela linha de comando, através de argv[1], que contém o caminho do arquivo informado pelo usuário.

### 2. O que acontece quando nenhum arquivo é informado?

A condição argc < 2 detecta a ausência do arquivo, exibe uma mensagem de erro e encerra o programa com EXIT\_FAILURE.

### 3. Qual função lê o arquivo?

A função read\_file(argv[1]), que lê o arquivo e retorna seu conteúdo para source\_code.

### 4. Qual função realiza a análise léxica?

A função lex(source\_code), que transforma o código-fonte em uma lista de tokens.

### 5. Qual função constrói a AST?

A função parse(&tokens), que transforma os tokens em uma Árvore Sintática Abstrata (AST).

### 6. Qual função executa a AST?

A função interpret(ast), que interpreta e executa a AST.

## **7. Em que ordem essas funções são chamadas?**

A ordem é: `read_file()` → `lex()` → `parse()` → `interpret()`.

### **Trecho do main.c:**

```
char* source_code = read_file(argv[1]);  
TokenList tokens = lex(source_code);  
ASTNode* ast = parse(&tokens);  
interpret(ast);
```

O trecho resume o fluxo principal: o arquivo é lido, convertido em tokens, transformado em AST e, por fim, interpretado.

## **Etapa 2 - Análise da leitura do arquivo**

### **Por que é necessário reservar memória?**

Para armazenar o conteúdo do arquivo durante a execução. `malloc(length + 1)` reserva o espaço necessário, incluindo um byte para `\0`.

### **Qual é o papel do caractere `\0`?**

Indicar o fim da string, permitindo que o conteúdo lido seja tratado corretamente como texto em C.

### **O que acontece quando o arquivo não existe?**

`fopen()` retorna nulo, `perror()` exibe o erro e `exit(EXIT_FAILURE)` encerra o programa.

### **Quem é responsável por liberar a memória reservada?**

A função `main()`, através de `free(source_code)`.

### Etapa 3 — Análise léxica

```
lucasdantas@HPdeLucasRD:~/mini-c-compiler$ ./mini-c testes/01_lexico_valido.txt
Source code:
let valor1 = 123 + 4;
print(valor1);

Tokens:
LET
IDENT(valor1)
EQUAL
NUMBER(123)
PLUS
NUMBER(4)
SEMICOLON
PRINT
IDENT(valor1)
SEMICOLON
EOF

AST:
AST_ASSIGN(valor1)
  AST_BINARY_OP(+)
    AST_NUMBER(123)
    AST_NUMBER(4)

Program output:
127
```

No teste válido, o lexer reconheceu `let` e `print` como palavras reservadas, `valor1` como identificador, `123` e `4` como números e os símbolos como seus respectivos tokens. O token `EOF` indica o fim da entrada.

```
lucasdantas@HPdeLucasRD:~/mini-c-compiler$ ./mini-c testes/02_lexico_invalido.txt
Source code:
let valor = 10 @ 2;

Unknown character: @
lucasdantas@HPdeLucasRD:~/mini-c-compiler$ |
```

Ao executar o programa, foi apresentada a seguinte mensagem:

**No teste inválido, foi obtido:**

**Mensagem:** Unknown character: @

**Caractere:** @

**Arquivo:** src/lexer.c

**Função:** lex(const char\* source)

**Classificação:** erro léxico, pois @ não é reconhecido pelo lexer.

**Como o lexer diferencia `let` de um identificador comum?**

Usa `strcmp()` para comparar a palavra. `let` gera `T_LET`, `print` gera `T_PRINT` e os demais nomes geram `T_IDENTIFIER`.

### Como um número com vários dígitos é construído?

isdigit() identifica os dígitos e o valor é construído por  $value = value * 10 + (source[i] - '0')$ .

### Identificadores como nota1 são aceitos?

Sim. Devem começar com letra e podem continuar com letras ou números.

### Identificadores iniciados por número são aceitos?

Não como um único identificador. 1nota, por exemplo, é separado em 1 e nota.

### O lexer registra linha e coluna?

Não. Os tokens não armazenam informações de linha ou coluna.

### Existe limite para a quantidade de tokens?

Sim. São reservadas 128 posições com `malloc(128 * sizeof(Token))` e não há redimensionamento.

### O que pode ocorrer se o programa gerar mais de 128 tokens?

Pode ocorrer escrita fora da memória reservada, causando comportamento indefinido, corrupção de dados ou segmentation fault. O `T_EOF` também ocupa uma dessas posições.

## Etapa 4 — Análise sintática

```
Source code:
let resultado = (10 + 5) * 2;
print(resultado);
```

```
Tokens:
LET
IDENT(resultado)
EQUAL
NUMBER(10)
PLUS
NUMBER(5)
MULT
NUMBER(2)
SEMICOLON
PRINT
IDENT(resultado)
SEMICOLON
EOF
```

```
AST:
AST_ASSIGN(resultado)
  AST_BINARY_OP(*)
    AST_BINARY_OP(+)
      AST_NUMBER(10)
      AST_NUMBER(5)
    AST_NUMBER(2)
```

```
Program output:
30
```

### 03\_sintatico\_valido.txt

**Resultado esperado:** execução correta, resultado 30.

**Resultado observado:** 30.

**Mensagem:** nenhuma mensagem de erro.

**Posição:** não se aplica.

**Função responsável:** parse\_statement() e parse\_expression() realizam a análise.

```
Source code:
let resultado 10 + 5;

Tokens:
LET
IDENT(resultado)
NUMBER(10)
PLUS
NUMBER(5)
SEMICOLON
EOF
Syntax error: expected '=' at pos=2
```

**04\_sem\_igual.txt**

**Resultado esperado:** erro sintático pela ausência de =.

**Resultado observado:** erro sintático.

**Mensagem:** Syntax error: expected '=' at pos=2

**Posição:** 2.

**Função responsável:** parse\_statement().

```
Source code:
let resultado = 10 + 5

Tokens:
LET
IDENT(resultado)
EQUAL
NUMBER(10)
PLUS
NUMBER(5)
EOF
Syntax error: expected ';' at pos=6
```

**05\_sem\_ponto\_virgula.txt**

**Resultado esperado:** erro sintático pela ausência de ;.

**Resultado observado:** erro sintático.

**Mensagem:** Syntax error: expected ';' at pos=6

**Posição:** 6.

**Função responsável:** parse\_statement().

```
Source code:
print((10 + 5);

Tokens:
PRINT
NUMBER(10)
PLUS
NUMBER(5)
SEMICOLON
EOF
Syntax error: expected ')' at pos=7
```

#### 06\_parentese\_incompleto.txt

**Resultado esperado:** erro sintático pela ausência de ).

**Resultado observado:** erro sintático.

**Mensagem:** Syntax error: expected ')' at pos=7

**Posição:** 7.

**Função responsável:** parse\_expression().

### Etapa 5 — Análise da AST

```
AST_ASSIGN(x)
  AST_BINARY_OP(+)
    AST_NUMBER(5)
    AST_NUMBER(3)
```

#### Qual nó representa a atribuição?

AST\_ASSIGN(x) representa a atribuição do valor à variável x.

#### Onde o nome x é armazenado?

No campo name do nó AST\_ASSIGN.

#### Quais nós representam os números?

AST\_NUMBER(5) e AST\_NUMBER(3).

#### Qual nó representa a soma?

AST\_BINARY\_OP(+).

#### Como left e right são utilizados?

Em uma operação binária, left aponta para o operando da esquerda e right para o da direita. Na soma, apontam para 5 e 3, respectivamente.

#### Representação da árvore:

```
AST_ASSIGN(x)
├── AST_BINARY_OP(+)
│   ├── left: AST_NUMBER(5)
│   └── right: AST_NUMBER(3)
```

## Etapa 6 — Tabela de símbolos e interpretação

```
Source code:
let idade = 30;
print(idade);

Tokens:
LET
IDENT(idade)
EQUAL
NUMBER(30)
SEMICOLON
PRINT
IDENT(idade)
SEMICOLON
EOF

AST:
AST_ASSIGN(idade)
  AST_NUMBER(30)

Program output:
30
```

### Variável definida

**Entrada:** let idade = 30; print(idade);

**Resultado observado:** 30.

**Classificação:** execução correta.

```
Source code:
print(idade);

Tokens:
PRINT
IDENT(idade)
SEMICOLON
EOF

AST:
AST_PRINT
  AST_VAR(idade)

Program output:
Runtime error: undefined variable 'idade'
```

### Variável não definida

**Entrada:** print(idade);

**Mensagem:** Runtime error: undefined variable 'idade'

**Classificação:** erro semântico detectado durante a execução.

```
Source code:
let resultado = 10 / 0;
print(resultado);

Tokens:
LET
IDENT(resultado)
EQUAL
NUMBER(10)
DIV
NUMBER(0)
SEMICOLON
PRINT
IDENT(resultado)
SEMICOLON
EOF

AST:
AST_ASSIGN(resultado)
  AST_BINARY_OP(/)
    AST_NUMBER(10)
    AST_NUMBER(0)

Program output:
Runtime error: division by zero
```

#### **Divisão por zero**

**Entrada:** let resultado = 10 / 0; print(resultado);

**Mensagem:** Runtime error: division by zero

**Classificação:** outro erro de execução.

#### **Por que a variável indefinida só é detectada pelo interpretador?**

Porque o parser verifica apenas se a estrutura sintática está correta. A existência da variável é verificada durante a execução, quando `lookup_symbol()` procura seu nome na tabela de símbolos.

#### **Etapa 7 — Compilador ou interpretador?**

O projeto é atualmente um interpretador, pois não gera assembly, bytecode, código C ou outro arquivo executável. `parse()` constrói a AST e `interpret()` a percorre diretamente, usando `exec_statement()` e `eval_expression()` para executar os comandos e calcular seus resultados.

## **Etapa 8 — Investigação de precedência e associatividade**

**Teste A — `print(2 * 3 + 4);`**

**Resultado obtido:** 14

**Esperado:** 10

```
AST_PRINT
  AST_BINARY_OP(*)
    AST_NUMBER(2)
  AST_BINARY_OP(+)
    AST_NUMBER(3)
    AST_NUMBER(4)
```

O parser interpretou a expressão como  $2 * (3 + 4)$ . Portanto, há problema de precedência, pois a soma foi agrupada antes da multiplicação.

**Teste B — `print(10 - 3 - 2);`**

**Resultado obtido:** 9

**Esperado:** 5

```
AST_PRINT
  AST_BINARY_OP(-)
    AST_NUMBER(10)
  AST_BINARY_OP(-)
    AST_NUMBER(3)
    AST_NUMBER(2)
```

O parser interpretou como  $10 - (3 - 2)$ . Há problema de associatividade, pois a subtração deveria ser associada da esquerda para a direita.

**Teste C — `print(2 + 3 * 4);`**

**Resultado obtido:** 14

**Esperado:** 14

```
AST_PRINT
  AST_BINARY_OP(+)
    AST_NUMBER(2)
  AST_BINARY_OP(*)
    AST_NUMBER(3)
    AST_NUMBER(4)
```

O parser interpretou como  $2 + (3 * 4)$ . Nesse caso, o resultado está correto, mas isso ocorre pela forma recursiva como a expressão foi agrupada, não porque o parser implemente corretamente todas as regras de precedência.

## **Etapa 9 — Modificação individual obrigatória**

### **Problema ou limitação escolhida:**

A linguagem não possuía suporte ao operador de resto %.

### **Comportamento anterior:**

O caractere % não era reconhecido pelo lexer e gerava erro léxico.

### **Comportamento desejado:**

Permitir expressões com %, como `10 % 3`, retornando o resto da divisão.

### **Arquivos alterados:**

`include/lexer.h`, `src/lexer.c`, `src/parser.c` e `src/interpreter.c`.

### **Funções alteradas:**

`lex()`, `print_tokens()`, `parse_expression()` e `eval_expression()`.

### **Decisões de implementação:**

Foi criado o token `T_MOD`, adicionado o reconhecimento de % no lexer, incluído o operador no parser e implementado seu cálculo no interpretador.

### **Caso de teste:**

```
let resto = 10 % 3; print(resto);
```

### **Resultado obtido:**

O lexer reconheceu `MOD`, a AST apresentou `AST_BINARY_OP(%)` e a saída foi `1`, como esperado.

### **Limitações que permaneceram:**

O parser ainda apresenta limitações de precedência e associatividade entre operadores.

| ID  | Entrada/Teste                        | Esperado                | Obtido                          | Situação  |
|-----|--------------------------------------|-------------------------|---------------------------------|-----------|
| T01 | let valor1 = 123 + 4; print(valor1); | 127                     | 127                             | OK        |
| T02 | let valor = 10 @ 2;                  | Erro léxico             | Unknown character: @            | OK        |
| T03 | let resultado = (10 + 5) * 2;        | 30                      | 30                              | OK        |
| T04 | let resultado 10 + 5;                | Erro por falta de =     | Erro em pos=2                   | OK        |
| T05 | let resultado = 10 + 5               | Erro por falta de ;     | Erro em pos=6                   | OK        |
| T06 | print((10 + 5);                      | Erro por falta de )     | Erro em pos=7                   | OK        |
| T07 | let idade = 30; print(idade);        | 30                      | 30                              | OK        |
| T08 | print(idade);                        | Variável indefinida     | Runtime error                   | OK        |
| T09 | let resultado = 10 / 0;              | Divisão por zero        | Runtime error                   | OK        |
| T10 | Precedência/associatividade          | Aritmética convencional | Resultados divergentes em A e B | Limitação |

**Limitações encontradas:** o projeto apresenta algumas limitações. O parser não implementa corretamente todas as regras de precedência e associatividade dos operadores. A lista de tokens possui capacidade fixa para 128 elementos e não é redimensionada dinamicamente. A tabela de símbolos também possui limite de 128 variáveis. Além disso, o lexer não registra linha e coluna nos tokens, reduzindo a precisão das mensagens de erro.

**Conclusão:** a análise permitiu compreender o fluxo completo do Mini C Compiler, desde a leitura do arquivo-fonte até sua interpretação. Foram analisados o lexer, parser, AST, tabela de símbolos e interpretador, além de identificadas limitações de precedência, associatividade e armazenamento. Como melhoria individual, foi implementado o operador de resto %, reconhecido pelo lexer, parser e interpretador, produzindo corretamente o resultado esperado.

**Referências:** Repositório original: ironrinox/mini-c-compiler. Roteiro da Atividade Individual Análise do Mini C Compiler.